

Instituto Politécnico de Lisboa
Instituto Superior de Engenharia de Lisboa

Solving the Uncapacitated Exam Timetabling Problem in Prolog
Using a Tournament-Selection Evolutionary Algorithm with Elitism

OCTOBER 2024

Author:

ADAM SZOKALSKI 53153

Course:

LOGIC AND COMPUTATION

Contents

1	Introduction	2
2	Problem Formulation	2
3	Problem Representation	2
4	The Solution	3
4.1	Crossover method	3
4.2	Mutation method	4
5	The Implementation	4
6	Results	4
7	Conclusions	7

1 Introduction

The Uncapacitated Exam Timetabling Problem (UETP) involves assigning n exams to s time slots such that scheduling conflicts are minimized. Conflicts arise when exams with overlapping student enrollments are assigned to the same or adjacent time slots.

This project suggests a solution based on a Evolutionary Algorithm (EA) used to find the most optimal solution in a feasible search space. Feasibility is provided by the use of Constraint Logic Programming (CLP) in a Finite Domain (FD).

The solution is inspired by these two papers: *A Shuffled Complex Evolution Based Algorithm for Examination Timetabling - Benchmarks and a New Problem Focusing Two Epochs*[1], *Solving an uncapacitated exam timetabling problem instance using a hybrid NSGA-II*[2]. While some methods overlap,, a different approach is proposed.

The final solution is implemented using SWI-Prolog and available [here](#).

2 Problem Formulation

Given the following:

- *number of courses* M , where each course corresponds to 1 exam,
- *number of periods* N (time slots),
- *conflict matrix* C , where C_{ij} represents the number of students enrolled both for courses i and j .

Produce an optimal solution satisfying a *hard constraint*:

H1: There cannot exist students sitting for more than one exam simultaneously.

and a *soft constraint*:

S1: The exams taken by one student have to be spread out evenly across the timetable.

The soft constraint **S1** is a modified version of **S1** from the papers mentioned. Originally it stated *Exams should be spread out evenly through the timetable*. However, in this project the goal has been shifted to ensure the most optimal solution for students. It ensures that each student has time to prepare before each exam.

3 Problem Representation

Each solution X is a candidate timetable, such as:

$$X = \{x_1, x_2, \dots, x_M\}, \quad \text{where } x_i \in [1, N] \quad \text{for } i = 1, 2, \dots, M.$$

The value of X_i represents the period in which the course i exam will be held.

Under this representation, the *hard constraint* **H1** takes the following form:

$$\mathbf{H1:} \quad X_i \neq X_j \quad \text{if } C_{ij} > 0.$$

X_i and X_j must be distinct (exams i and j can't be scheduled in the same period) if the number of overlapping students C_{ij} is greater than 0.

The *soft constraint* **S1** becomes an optimization problem:

$$\mathbf{S1}: X^* = \arg \min_X \sum_{i,j=1}^M \text{Proximity}(i, j)$$

where the $\text{Proximity}(i, j)$ function is given by:

$$\text{Proximity}(i, j) = \begin{cases} C_{ij} \cdot P(t_i, t_j), & \text{if } C_{ij} > 0 \\ 0, & \text{if } C_{ij} = 0 \end{cases}$$

$$P(t_i, t_j) = \begin{cases} 2^{(\epsilon - |X_i - X_j|)}, & \text{if } 1 \leq |X_i - X_j| < \epsilon \\ 1, & \text{otherwise} \end{cases}$$

With a hyperparameter ϵ empirically set to $\epsilon = 5$.

The optimization goal is to minimize the total pairwise proximity of exams with respect to the number of students enrolled in both (C_{ij}). The exponential form of the penalty speeds up the convergence. The hyperparameter ϵ represents an ideal distance between such exams.

4 The Solution

The problem represented in the fore-mentioned form is an optimization problem within a feasible search-space defined by **H1** and a goal represented by **S1**.

Since the solution is bound by constraints many methods would converge into a local optima. For that reason an *Evolutionary Algorithm* has been selected. By nature EAs are more focused on *exploration* than *exploitation* of the data hence this method is more likely to yield better results than others.

The *fitness function* has been set to the inverse of the minimization goal so the problem becomes a maximization problem. A *zero-division* case has been taken into account so the final fitness formula becomes:

$$\text{Fitness}(i, j) = \frac{1}{1 + \sum_{i,j=1}^M \text{Proximity}(i, j)}$$

The chosen variation of the Evolutionary Algorithm is as follows:

1. Initialization - a population is initialized with 8 random and feasible specimen (X),
2. Tournament Selection - promotes exploration,
3. Crossover - experimental crossover method ensuring children feasibility (rate = 0.8),
4. Mutation - experimental mutation method ensuring mutated feasibility (rate = 0.5),
5. Elitism - preserves the best solutions. Small rate of 0.1 ensures exploration.
6. Termination after 100 generations

4.1 Crossover method

The crossover method works according to this algorithm:

1. Parents X and Y
2. Choose a random number $p \in [1, M/2]$.

3. p times:

- (a) Randomly choose a number $q \in [1, M]$
- (b) Swap X_q and Y_q
- (c) Check feasibility
- (d) Commit changes if feasible

4.2 Mutation method

The mutation method works according to this algorithm:

1. Specimen X .
2. Until commits feasible:
 - (a) Randomly choose a number $q \in [1, M]$
 - (b) Randomly choose a number $m \in [1, N]$
 - (c) Set $X_q = m$
 - (d) Check feasibility
 - (e) Commit changes if feasible

5 The Implementation

The implementation is available on GitHub [here](#).

SWI-Prolog was the language of choice. **CLP(FD)** was used to ensure the *hard constraint H1* in the following way:

```
specimen(Schedule) :-
    config(course_number, NumCourses),
    config(slots, NumSlots),
    length(Schedule, NumCourses),
    Schedule ins 1..NumSlots,
    all_distinct_if_conflicting(Schedule),
    random_between(111, 999, Seed),
    labeling([random_variable(Seed)], Schedule).

all_distinct_if_conflicting(Schedule) :-
    findall((I, J), (conflict(I, J), I < J), ConflictPairs),
    maplist(enforce_distinct_pair(Schedule), ConflictPairs).

enforce_distinct_pair(Schedule, (I, J)) :-
    nth1(I, Schedule, SlotI),
    nth1(J, Schedule, SlotJ),
    SlotI #\= SlotJ.
```

The `specimen(Schedule)` predicate was used in the implementation to ensure feasibility using *constraint propagation*.

6 Results

The resulting optimal timetable ($M = 80$, $N = 20$) achieved a fitness score of 14.72.

Course	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
SET										x										
AA		x																		
ACir							x													
ACp								x												
AED		x																		
AIEC	x																			
ALGA																x				
AM1														x						
AM2									x											
Ant					x															
ASI													x							
AVE						x														
BD									x											
CAC					x															
CCD			x																	
CG					x															
CGAV					x															
CMov						x														
Com																	x			
Cpl													x							
CSDin				x																
CSDist				x																
CSI	x																			
CSM									x											
CSO					x															
E1			x																	
E2												x								
EA				x																
EGP															x					
Elctr			x																	
ES										x										
F1		x																		
F2								x												
FAE		x																		
FT						x														
GSI			x																	
IRS			x																	
IS		x																		
ITI	x																			
LC										x										
LSD					x															
M1				x																
M2	x																			
MAT	x																			
MNO			x																	
MSr						x														
OE											x									
OGE					x															
PC									x											
PCI																x				
PDSr						x														
PE																				
PF		x																		
Pg																				
PI											x									
PICC/CPg											x									
PIV				x																

Course	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
POO																			x	
PR					x															
PSC													x							
RCom	x																			
RCp			x																	
RDC	x																			
RI																		x		
RSCM																	x			
SCDig							x													
SCDist			x																	
SCO								x												
SD																				x
SE1								x												
SEAD1																	x			
SG															x					
SI				x																
SI1														x						
SI2												x								
SOi							x													
SOt		x																		
SS										x										
ST									x											
STBL		x																		

7 Conclusions

The algorithm converged in the 14th generation which is rather fast. Tweaking the hyperparameters could possibly produce better results. A larger population size could also allow more exploration however it would be very computationally expensive.

For smaller values of N the constraint becomes very strict and generating feasible solutions takes a long time because the constraint propagation has to reconsider many choices made.

During experiments the smallest number of periods N that converged in an acceptable time (less than 10min) was $N = 20$. It's worth keeping in mind that the computations were made using a consumer grade laptop.

References

- [1] Nuno Leite, Fernando Melicio, and Agostinho Rosa. A shuffled complex evolution based algorithm for examination timetabling - benchmarks and a new problem focusing two epochs. *ECTA 2014 - Proceedings of the International Conference on Evolutionary Computation Theory and Applications*, pages 112–124, 01 2014.
- [2] Nuno Leite, R. Neves, Nuno Horta, Fernando Melicio, and Agostinho Rosa. Solving an uncapacitated exam timetabling problem instance using a hybrid nsga-ii. *IJCCI 2012 - Proceedings of the 4th International Joint Conference on Computational Intelligence*, pages 106–115, 01 2012.